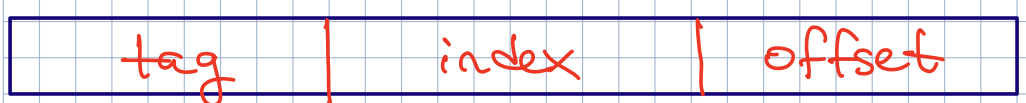
Cache Memory

Total # of bits needed for a cache is a function of cache size & address size

* 32-bit address
* Direct-mapped cache
* Cache size is 2^n blocks (n-bit for index)
* Block size is 2^m words (2^{m+2} bytes)
  (m-bits word within block, 2 bits byte port of the addr)

Size of tag field: $32-(n+m+2)$



n-bits

block offset + byte offset

$(m+2)$ bits

$32-(n+m+2)$ bits

Total # of bits in a direct-mapped cache

$= 2^n \times$ (block size + tag size + valid field size)

$$= 2^n \times \left(2^m \times 32 + (32 - n - m - 2) + 1 \right)$$

$$= 2^n \times (2^m \times 32 + 31 - n - m)$$

Slide 21

Block number = Block addr mod (# of blocks in cache)

$$\text{Block addr} = \frac{\text{byte addr}}{\text{bytes per block}}$$

$$\text{Block addr} = \lfloor 1200/16 \rfloor = 75$$

Block number = 75 mod 64 = 11

Measuring Cache Performance

CPU time = (CPU exec. clock cycles + Memory stall clock cycles) × CCT

↓

(primarily from cache misses)

[(Combining read & writes by using a single miss rate and miss penalty)]

$$\text{Memory Stall cycles} = \frac{\text{Mem access}}{\text{program}} \times \text{Miss rate} \times \text{Miss penalty}$$

$$\text{Memory stall cycles} = \frac{\text{Instructions}}{\text{program}} \times \frac{\text{Misses}}{\text{Instr.}} \times \text{Miss penalty}$$

Example: (from textbook — Page 400)

I-cache & D-cache

I-cache

Miss rate = 2% ×

Miss penalty = 100 cc

D-cache

Miss rate = 4%

Miss penalty = 100 cc

Base CPI (without memory stalls) = 2

Lw / Sw = 36 % instructions

Question: Determine how much faster a processor would run with a perfect cache that never missed?

Instr miss cycles = $I \times 2\% \times 100$

$= 2I$ cc.

Data Miss cycles = $(I \times 36\%) \times 4\% \times 100$

$= 1.44\ I$

Total # of stall cycles = $3.44\ I$

Total $CPI_{stall} = 2 + 3.44 = 5.44$

$$\frac{\text{CPU time with stalls}}{\text{CPU time with perfect cache}} = \frac{I \times CPI_{stall} \times CCT}{I \times CPI_{perfect} \times CCT}$$

$$= \frac{5.44}{2} = 2.72$$

(Slide 32)

$$AMAT = \left(\text{Hit time} \times \text{Hit ratio}\right) + \left(\text{Miss time} \times \text{Miss ratio}\right)$$

$$AMAT = \left(1_{\text{cycle}} \times 0.95\right) + \left([1+20] \times 0.05\right)$$

$$= 0.95 + 21 \times 0.05 = 2$$

$$\text{Exec Time} = \left(\text{Exec cycles} + \text{Stall cycles}\right) \times \text{Cycle Time}$$

$$\text{Stall cycles} = \#\text{ of Instr} \times \text{Miss rate} \times \text{Miss penalty}$$

To improve performance

- Decrease miss ratio
  - (More flexible placement of blocks) ✓
- Decrease Miss penalty
  - (Use multi-level Cache)

1. A block can go in exactly one place → Direct mapped
2. A block can be placed in any location → fully-associative
3. Middle range → set-associative

Set Associative

There are fixed # of locations where each block can be placed.

"n" locations → n-way set assoc.

- Each block in a memory maps to
  - * unique set in cache given index field
  - * block can be placed in any element of that set

Block in 4-diff loc. ⇒ 4-way

Set Associative:

→ (block number) mod (# of sets in cache)

Direct Mapped

(block number) mod (# of cache blocks)

Direct-mapped → 1-way set assoc.
cache (each set has 1 block)

Fully-assoc. with m blocks → m-way set assoc. (one set with m-blocks)

In set assoc cache ⇒ which entry to replace?
Need eg placement

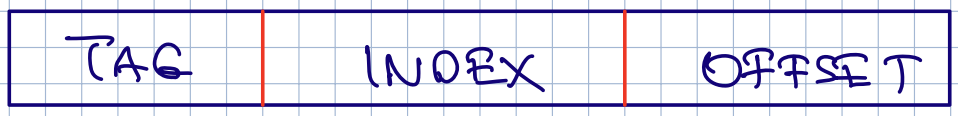
LRU (Least Recently used)

- The block replaced is the one that has been unused for the longest time

Locating a Block in a Set-Assoc. Cache

Just in direct-map, index value is used to select set.

Tag of every cache block within the set is checked to see if it matches the block address from the CPU. (All tags are searched parallel)



If total size is same,

↑ associativity ⇒ ↑ of blocks per bit

factor of two ↑ in assoc ⇒
- ↓ size of index by 1 bit
- ↑ size of tag by 1 bit

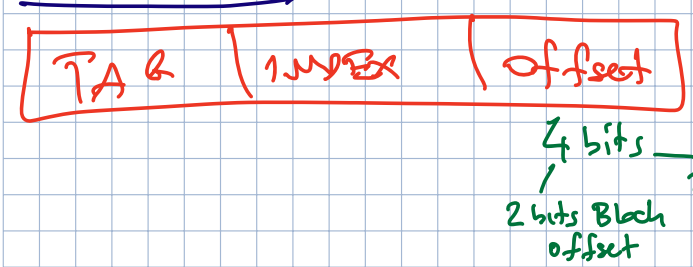
If fully assoc ⇒ index = ∅ bits

Example: (Page 409)

Size of Tags versus Set Associativity

Increasing associativity requires more comparators and more tag bits per cache block. Assuming a cache of 4096 blocks, a 4-word block size, and a 32-bit address, find the total number of sets and the total number of tag bits for caches that are direct mapped, two-way and four-way set associative, and fully associative.

SOLUTION



4K blocks, 4-word block size & 32-bit addr.

2) Direct

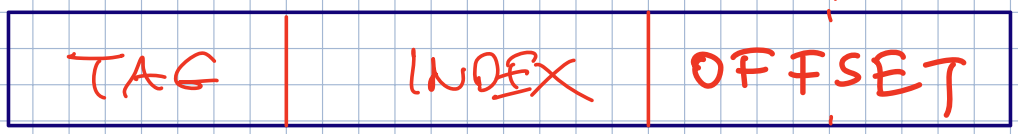
4 word blocks = 2^{4} bytes per block. (4 → # of bits offset)

Block size

Block offset = 4 bits (block + byte)

$4K = 2^{12}$ blocks

Index = 12 bits



(block offset + byte offset)

16 bits 12 bits 4 bits

TAG size (total) = 16 bits × 4096 = 66K tag bits

---

b) Four WAY

4096 blocks, 4-way

word blocks since

Index ⇒ #of sets = $\frac{4K}{4}$ = 1K

#of sets = 2^{10}



18 bits 10 bits 4 bits

TAG size (total) = 18 bits × 1K sets × 4 4-ways

= 72 × 1024 = 74K TAG bits

---

c) Fully Assoc.

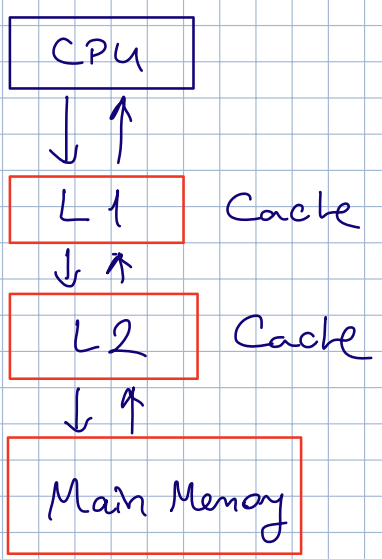
TAG	INDEX	BLOCK offset
28 bits	0 bits	4 bits

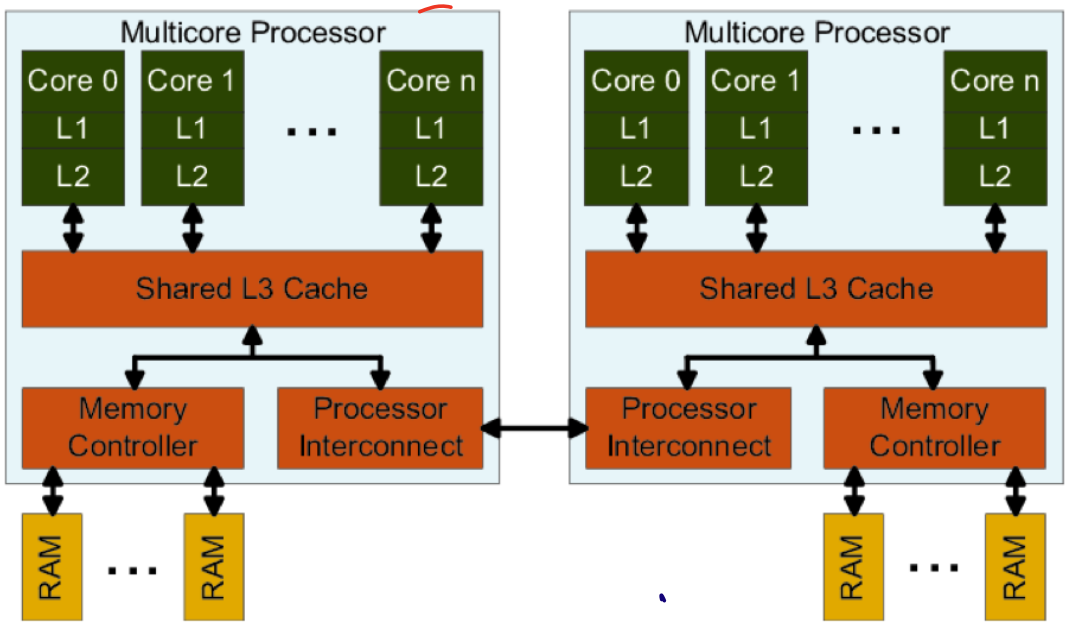
TAG size (total) = 28 x 1 x 4K

set

= 28 x 4096 = 115K bytes

Multi-level Caches





Schematic Representation of Contemporary Multicore Architectures

Example: (Problem given in Page 410)

Performance of Multilevel Caches

Suppose we have a processor with a base CPI of 1.0, assuming all references hit in the primary cache, and a clock rate of 4 GHz. Assume a main memory access time of 100 ns, including all the miss handling. Suppose the miss rate per instruction at the primary cache is 2%. How much faster will the processor be if we add a secondary cache that has a 5 ns access time for either a hit or a miss and is large enough to reduce the miss rate to main memory to 0.5%?



(1 Hz = 1 cycle/sec)

$4\ GHz = 4 \times 10^9\ Hz = 4 \times 10^9$ cycles/sec.

$\frac{1}{4 \times 10^9}$ sec/cycles $= 0.25 \times 10^{-9}$ sec/cycle

$= 0.25$ ns/clock cycle

Miss penalty to main memory $= \frac{100\ ns}{0.25\ ns/\text{clock cycle}} = 400$ clock cycles

Total CPI = Base CPI + Memory stall cycles / instr

$= 1.0 + 2\% \times 400 = 9$

CPI = 9 for one-level cachy

2-level = a miss in first level can be satisfied either by second-level or by main memory.

$\frac{5\ ns}{0.25\ ns/\text{clock cycle}} = 20$ clock cycles

Total CPI = 1 + Primary stalls per Inst + Secondary stalls per Instr

$= 1 + 2\% \times 20 + 0.5\% \times 400$

$= 1 + 0.4 + 2.0 = 3.4$

Secondy code is faster by $9/3.4 = 2.6$

Alternate way

$= 1 + ((2\% - 0.5\%) \times 20) +$

$(0.5\% \times (20 + 400))$

$= 1 + 0.3 + 2.1 = 3.4$